



LINGUISTICS WITH INKHAVEN

Linguistic Research with Inkhaven

Лингвистическое исследование с Inkhaven

Analysing a Living Language — a Study of Russian

Vladimir Ulogov

2026 · examples assume Inkhaven 1.8.2 or newer

Contents

Part I – The Workbench	2
1 – A workbench for real languages	3
The shape of the workflow	4
Why model Russian?	4
What you will build	5
2 – Modelling Russian	6
Creating the language	6
The inventory: sounds, or letters?	7
A starter lexicon	8
Part II – Sound	10
3 – The sound system	11
The shape of the inventory	11
Counting the sounds	12
The contrasts Russian leans on: minimal pairs	13
Where the IPA shows through	13
Building a phonemic model in the IPA	14
Before and after	14
From underlying to surface: allophony	15
Stress and the metre of verse	17
Where each sound lives	18
Part III – Words	19
4 – Morphology and glossing	20
A fragment of the noun declension	21
Reading a form backwards	22
Interlinear glossing	22
Agreement	23
Part IV – Sentences	25
5 – Syntax	26
A baseline order	27
The structure of a clause	27
Moving a piece	28
Checking a clause	28
Part V – Usage	30
6 – A corpus of Russian	31

Assembling the corpus	32
What is frequent?	33
A word in its contexts	34
The company a word keeps	34
Where a word lives in the text	35
Part VI – Structure and History	37
7 – Grammar and typology	38
The typological profile	39
What each verb demands: valence	40
Deeper parameters	40
Checking the description	41
Testing against the universals	42
8 – Historical linguistics	44
A famous change	45
Testing the consequences	45
Russian among the Slavic languages	46
Appendix A – The research workflow	48
Modelling the language	48
The sound system	49
Words and sentences	49
The corpus	50
Typology and history	50
The companion	51
About the Author	52
A work of love	53
A note on cooperation	53
Where to find more	54

Before we begin

Inkhaven began life as a tool for **inventing** languages, and it is still very good at it — designing a sound system, growing a lexicon, declining and conjugating, evolving a family, printing the grammar. But over its 1.7 series it grew a full linguistic layer — glossing, corpus tools, a syntax engine, a typological questionnaire, a hypothesis register — and somewhere along the way it stopped being a conlang tool that could describe languages and became a **general linguistics workbench** that happens to be excellent at conlang. The machinery never cared whether the language it was handed was imaginary or real: a phoneme inventory is a phoneme inventory; a corpus is a corpus; an interlinear gloss glosses whatever you give it. Point the tools at a language that already exists, and you have a place to model it explicitly and interrogate the model — a laboratory for real linguistics that also, when you want it, builds worlds.

This book is the mirror image of its companion, *Developing a Constructed Language*. That book uses the workbench to run from nothing to a finished invented language. This one takes a language you did not invent — **Russian** — and shows how to study it with the very same commands: measure its sound system, gloss its sentences, parse its rich morphology, draw the structure of its clauses, build a corpus from real texts, place it on the typological map, and reason about its history. Russian is the running example on every page; the **methods** are the point, and they transfer to any language you can write down — invented or inherited.

You do not need to speak Russian to follow along — every example is translated — and you do not need a linguistics degree, because each idea is defined where it first appears. What you do need is the willingness to treat a language as an object of study: to state a hypothesis about it and let the evidence answer.

A word on honesty. Modelling a real language in a tool built for invented ones has edges, and this book does not hide them — where Inkhaven’s assumptions (that phonemes are written in the IPA, say) meet Russian’s reality, we will stop and look, because those seams are often where the linguistics is.

PART I

The Workbench

CHAPTER 1

1

A workbench for real languages

Linguistics is an empirical science. A grammar is a set of claims about a language — “Russian has six cases”, “stress is unpredictable”, “adjectives agree with their nouns” — and each claim is answerable against data. What a working linguist needs is somewhere to write the claims down **formally**, apply them to real forms, and see whether they hold. That is exactly what Inkhaven’s language model gives you: a declared inventory, a lexicon, a morphology, a grammar, and a set of tools that read them and report.

The shape of the workflow

Studying a language in Inkhaven follows one arc, whatever the language:

1. **Model** the language — create it, declare its phoneme inventory, add a lexicon.
You are building an explicit, machine-readable description.
2. **Measure** the sound system — its inventory, its statistics, its minimal pairs.
3. **Analyse** words and sentences — parse the morphology, gloss real text, draw the syntax.
4. **Gather** usage — build a corpus from real texts and query it.
5. **Situate** the language — against typological universals, and against its own history.

Every step is a command, and every command reads the model you built and prints what it finds. Nothing is hidden in a black box: the phonology you declare is the phonology the tools use, so when a result surprises you, the fix is always to look at the model.

TERM Model (of a language)

An explicit, formal description — here, the phoneme inventory, lexicon, morphology and grammar you declare in Inkhaven. A model is deliberately a simplification; its value is that, being explicit, it can be **tested**. When the model and the language disagree, you have learned something.

Why model Russian?

Russian is a rewarding subject for a first study. Its sound system has the famous **hard/soft** (palatalization) contrast that doubles its consonants and fills the language with minimal pairs. Its morphology is **fusional** and rich — six cases, three genders, two numbers, verbal aspect — so there is a great deal for a morphological parser to chew on. Its word order is freer than English, which makes its syntax interesting. It has a deep, well-understood history of sound change. And it is written in Cyrillic, which is a useful test in itself: a workbench that only worked for the Latin alphabet would be no workbench at all.

A NOTE ON CYRILLIC

Every tool in this book handles Cyrillic text — the lexicon, the metrics, the glosser, the corpus. The one place the Latin bias of a shared reference table shows through is the distinctive-**feature** analysis of Chapter 3, and we meet it head-on there. Everywhere else, Russian is a first-class citizen.

What you will build

By the end you will have a working model of a slice of Russian — an inventory of its sounds, a lexicon of common words, a handful of glossed sentences, a small corpus drawn from a real text, and a register of hypotheses about its history — and, more importantly, a method you can turn on any language you like. The next chapter starts where every study starts: creating the language and giving it its sounds.

WHAT YOU LEARNED

- Inkhaven's language model — inventory, lexicon, morphology, grammar — is a formal, testable description; the same tools that build an invented language **analyse** a real one.
- The workflow is model → measure → analyse → gather → situate, each step a command that reads your model and reports.
- Russian is the running example: a palatalization-rich sound system, fusional morphology, free-ish word order, a deep history — and Cyrillic, which the toolset handles throughout.

CHAPTER 2

2

Modelling Russian

A study begins by making the object of study explicit. In Inkhaven that means creating the language, declaring its sounds, and giving it enough vocabulary to work with. The model you build here is the ground every later chapter stands on.

Creating the language

One command makes the language and its empty chapters — phonology, dictionary, grammar, and the rest:

```
inkhaven language init Russian
```

Russian now exists as a language book in your project, alongside any others. It is blank: no sounds, no words. We fill it in.

The inventory: sounds, or letters?

Here is the first real decision, and it is a linguistic one. Inkhaven's phoneme inventory is a list of segments, each with an `ipa` field. A phonologist's instinct is to fill it with the International Phonetic Alphabet — Russian's /p/, /pʲ/, /a/, /i/, and so on — because that is the level at which sound **contrasts** live, and the distinctive-feature tools of the next chapter are built on IPA symbols.

But Russian is written in Cyrillic, and much of what we will do — glossing real sentences, building a corpus from a real book — works over Cyrillic **text**. If our inventory were in the IPA, the tools could not line the inventory up with the words. So for this study we model Russian at the level of its **orthography**: each Cyrillic letter is a segment. It is a simplification — Cyrillic spelling is not a perfect phonemic transcription — but it keeps the model and the texts in the same alphabet, and it is honest about what it is.

TERM Orthography vs. phonology

Orthography is how a language is spelled; **phonology** is how it sounds. They are never identical — English **though**, **through**, **tough** share **ough** and share almost nothing in sound. Russian's fit is much closer than English's, but still imperfect: the letter **о** is pronounced **a** when unstressed. Modelling at the orthographic level trades some phonological precision for the ability to work directly with written Russian — a trade this book makes deliberately, and revisits in Chapter 3.

Open the Russian language's **Phonology** chapter in the editor and declare the inventory as an HJSON block — the ten vowel letters, the twenty-one consonants, and the two signs:

```
{ phonemes: [
  { ipa: "a", kind: "vowel" } { ipa: "e", kind: "vowel" }
{ ipa: "ë", kind: "vowel" }
  { ipa: "и", kind: "vowel" } { ipa: "o", kind: "vowel" }
{ ipa: "y", kind: "vowel" }
  { ipa: "ы", kind: "vowel" } { ipa: "э", kind: "vowel" }
{ ipa: "ю", kind: "vowel" }
  { ipa: "я", kind: "vowel" }
  { ipa: "б", kind: "consonant" } { ipa: "в", kind:
"consonant" } // ...и так далее...
  { ipa: "ь", kind: "consonant" } { ipa: "ъ", kind:
"consonant" }
] }
```

`inkhaven language stats Russian` then confirms what you declared:

```
inventory · 33 phonemes (23 C / 10 V)
```

Thirty-three segments — the Russian alphabet exactly. (The soft and hard signs, **ь** and **ъ**, are letters rather than sounds; classing them as consonants lets words that contain them segment cleanly, which is all the model needs of them.)

A starter lexicon

Analysis needs words. Add them one at a time, or import a list; each entry is a headword, a part of speech, and a gloss:

```
inkhaven language add-word Russian мать --type noun --
translation mother
inkhaven language add-word Russian брат --type noun --
translation brother
inkhaven language add-word Russian дом --type noun --
translation house
inkhaven language add-word Russian окно --type noun --
translation window
```

A handful is enough to begin; the corpus chapter will pull far more vocabulary from a real text. Everything is Cyrillic throughout — the headword, the bucket it files under (**мать** lands under **М**), the segmentation — and it simply works.

THE MODEL IS YOURS TO INSPECT

Every result in this book is computed from the inventory and lexicon you just declared — nothing else. If a later number surprises you, the answer is always in the model: add the missing word, fix the inventory, and run the command again. That transparency is the whole point of modelling a language explicitly.

WHAT YOU LEARNED

- `language init <name>` creates the language; its phoneme inventory is declared as an HJSON block in the Phonology chapter, and `stats` confirms it.
- We model Russian at the **orthographic** (Cyrillic-letter) level — a deliberate simplification that keeps the model and real Russian texts in one alphabet, at some cost in phonological precision (Chapter 3 revisits the cost).
- `add-word` builds the lexicon; Cyrillic headwords, buckets and segmentation all work directly.

PART II

Sound

CHAPTER 3

3

The sound system

With an inventory declared and a lexicon growing, we can ask the first empirical questions. How large and how balanced is Russian's sound system? What contrasts does it lean on? And where does a tool built around the IPA meet a language modelled in Cyrillic? This chapter is the workbench doing phonology.

The shape of the inventory

stats already told us the size — thirty-three segments, twenty-three consonants to ten vowels. That ratio is itself a finding: Russian is **consonant-heavy**, and the

reason is the hard/soft contrast. Most Russian consonants come in pairs, one **plain** and one **palatalized** (pronounced with the tongue raised toward the palate), and that doubling roughly halves the work the vowels have to do. In the orthography the contrast is carried partly by the consonant and partly by the **following vowel letter**: the pairs **а/я**, **о/ё**, **у/ю**, **э/е** and the sign **ь** are how Russian spelling writes “the consonant before me is soft.”

Counting the sounds

metrics reads the lexicon and reports the sound system quantitatively:

inkhaven language metrics Russian

entropy · 4.31 bits (max 5.04) · evenness 85% · perplexity 19.8
 zipf · slope -0.91 (≈ -1 is Zipfian) · fit R^2 0.94
 syllables · 41 attested / 210 possible · saturation 20%
 prosody · 1.9 moras/word · 62% heavy syllables

Entropy measures how evenly the language spreads its words across its sounds — high evenness means no handful of letters carries everything. The **Zipf** slope near -1 says the letter frequencies follow the same power law natural language always does (a reassuring sign the model behaves like a real language). **Saturation** asks how much of the possible syllable space the language actually uses — a low figure is normal; no language uses more than a fraction of what its phonotactics permit.

TERM Phoneme entropy

A measure, in bits, of how unpredictable the next sound is — highest when every sound is equally likely, lower when a few dominate. It captures the **balance** of an inventory: a language that leans hard on a handful of sounds has low entropy, one that uses its whole inventory evenly has high entropy.

The contrasts Russian leans on: minimal pairs

The sharpest tool for a sound system is the **minimal pair** — two different words that differ in exactly one sound. Each minimal pair proves that the one sound they differ in is **contrastive**: swapping it changes the word.

[inkhaven](#) language pairs Russian

Russian is famously full of them, and the richest source is exactly the hard/soft contrast:

examples:

мать / мять	а~я	(mother / to crumple)
был / бил	ы~и	(was / he beat)
дом / том	д~т	(house / volume)

мать / мять is the classic case: the words differ only in the vowel letter, а versus я, which in Russian spelling signals a hard т versus a soft т'. That one orthographic difference is a genuine phonemic contrast, and the language is built on hundreds like it.

TERM Minimal pair

Two words identical but for a single sound in the same position (**pat** / **bat**, **мать** / **мять**). Minimal pairs are the primary evidence that two sounds are separate phonemes in a language rather than variants of one — if swapping them can change a word, the language must distinguish them.

Where the IPA shows through

Ask the workbench to judge the inventory's **naturalness**, and the seam appears:

[inkhaven](#) language naturalness Russian


```
outside · а е ё и о у ... (not in the feature matrix)
score   · 0.35 (0–1; higher = more typologically ordinary)
```

The low score is not a verdict on Russian — Russian is as natural as a language gets. It is the tool telling the truth about its own limits. Naturalness, and the functional-load half of the minimal-pairs report, are computed from a table of **distinctive features** (voicing, place, manner) that is keyed to IPA symbols. Our segments are Cyrillic **letters**, which that table has never heard of — so it reports them “outside the feature matrix” and cannot grade them.

Building a phonemic model in the IPA

To make the feature tools work, give Russian a second inventory written in the International Phonetic Alphabet. The move is simple: each phoneme’s `ipa` field holds its IPA symbol, and its `romanize` field holds the Cyrillic letter you want it **shown** as. Declare it in the Phonology chapter of a parallel language (call it `RussianIPA`) just as before, but phonemically:

```
{ phonemes: [
  { ipa: "a", kind: "vowel",    romanize: "а" }
  { ipa: "ɪ", kind: "vowel",    romanize: "ы" }
  { ipa: "p", kind: "consonant", romanize: "п" }
  { ipa: "b", kind: "consonant", romanize: "б" }
  { ipa: "s", kind: "consonant", romanize: "с" }
  { ipa: "z", kind: "consonant", romanize: "з" }
  { ipa: "g", kind: "consonant", romanize: "г" }
  // ...the other vowels, consonants and the palatalized pairs...
] }
```

The `romanize` field is the bridge: the model reasons over the IPA symbol `p`, but any form it prints shows the Cyrillic `п`. You would transcribe the lexicon phonemically too — `дом` becomes `dom`, `был` becomes `bɪl` — so that words and inventory share the IPA level.

Before and after

Now ask the same question of the phonemic model:

inkhaven language naturalness RussianIPA

inventory	· 24 phonemes (18 C / 6 V) – typical
voicing	· p/b t/d s/z ʃ/ʒ
places	· labial coronal dorsal
missing	· w (near-universal)
score	· 0.94 (0–1; higher = more typologically ordinary)

The score jumps from 0.35 to **0.94**, and — more to the point — the analysis becomes real. Where the Cyrillic model could say nothing, the phonemic one recognizes that Russian’s obstruents come in neat voicing pairs (p/b, t/d, s/z, ʃ/ʒ), that its consonants cover the major places of articulation, and that it lacks only the near-universal w — a fair and accurate portrait of a thoroughly ordinary, thoroughly natural sound system. The minimal-pairs report changes too: **мать/мять** is no longer “outside the feature matrix” but a contrast in **palatalization**, and the functional load of each feature can finally be weighed.

TWO MODELS, TWO JOBS

This is the pattern for studying any language whose script is not the IPA: keep an **orthographic** model for the work that lives in the writing system — glossing, the corpus, syntax over real text — and a **phonemic** model in the IPA for the work that lives in the sounds — naturalness, distinctive features, functional load. Each is right for its half of the subject; the mistake is expecting one model to do both.

From underlying to surface: allophony

A phonemic model buys more than a naturalness score. It lets you write the language’s **rules** — the regular processes that reshape a word between the form stored in the mind and the form that reaches the mouth — and then **apply** them. The distinction is foundational: the **underlying** form is what the grammar builds, the **surface** form is what is pronounced, and the rules map one to the other.

Russian’s cleanest example is **final devoicing**: a voiced obstruent at the end of a word is pronounced voiceless. The word **друг** “friend” is built on an underlying

/g/, but said with a final [k]. You declare the rule in the phonology’s **allophony** block, in the standard notation *target > result / left _ right*:

```
{ allophony: [
  { name: "final devoicing", rule: "g > k / _ #" }
  { name: "final devoicing", rule: "d > t / _ #" }
  // ...one per voiced obstruent; `#` is the word edge
] }
```

Then ask Inkhaven to derive a word’s pronunciation:

```
inkhaven language ipa RussianIPA --word дpyг
```

```
underlying  /drug/
surface     [druk]
romanized   друк
```

The engine segments the word into phonemes, runs the ordered rules (underlying → surface), and shows both the surface IPA and its written rendering. **род** “year” surfaces as [got]; **дом** “house”, with no final obstruent to devoice, passes through unchanged. Three words, three correct predictions from one rule.

TERM **Allophony**

The rule-governed variation in how a phoneme is pronounced depending on its context — the underlying /g/ of **дpyг** surfacing as [k] word-finally. Allophonic rules are the synchronic (present-tense) processes of a living grammar, distinct from the **diachronic** sound changes of Chapter 8, which act across centuries. The same rewrite notation describes both.

RULES APPLY IN ORDER

Allophony rules fire in the order you declare them, each seeing the output of the last — the **feeding order** every phonology textbook teaches. If one rule palatalizes a consonant and a later rule only affects palatalized consonants, the ordering is what makes the second rule apply. Reorder the block and you can change the derivation, which is exactly the kind of hypothesis this lets you test.

Stress and the metre of verse

Russian has one more suprasegmental feature that no rule can predict: **stress**. Unlike Finnish (always initial) or Polish (always penultimate), Russian stress is **lexical** — it falls on a different syllable in different words, it **moves** within a word's paradigm, and it is **contrastive**: замо́к “castle” and замо́к “lock” differ in nothing but where the stress lands. Because no rule places it, you must **record** it — either with the lexicon's **stress** field (the stressed syllable, 1-based) when you add a word, or with an acute mark over the vowel in the text itself:

```
inkhaven language add-word зелёный --type adjective --translation
green --stress 2
```

With stress known, Inkhaven can **scan** a line of verse — mark each syllable strong or weak and name the metre. Take the most famous opening in Russian poetry, Pushkin's *Ruslan and Lyudmila*, with its stresses marked:

```
inkhaven language scan Russian --text "У лукомóрья дуб зелёный"
```

```
У | лу ко мор ья | дуб | зе лё ный
· | × × / × | / | × / ×
→ iambic tetrameter · 9 syllable(s) · fit 0.88
```

The line resolves to **iambic tetrameter** — four rising feet — the metre of the whole poem. Its ninth syllable is a **feminine ending**, an extra unstressed beat past the fourth foot, which is why nine syllables still scan as four feet. The little **у**, a monosyllable, is left **flexible** (**·**): a one-syllable word promotes or demotes to fit the metre, exactly as scansion by hand allows.

TERM Scansion

Marking the stressed and unstressed syllables of a line of verse and identifying the repeating **foot** they form — the **iamb** (weak–strong, **× /**), **trochee** (strong–weak), **dactyl**, **anapest**. A line of n feet is a dimeter, trimeter, tetrameter, and so on. Scansion is where phonology (stress) meets poetics, and it works only once each word's stress is known — which for Russian means the lexicon or the page, never a rule.

Where each sound lives

distribution reports where in the word each segment tends to appear — onset, nucleus, coda, word edges — and flags any with a restricted distribution:

inkhaven language distribution Russian

For Russian this surfaces real facts of the orthography: **ѣ** never begins a word and **ы** never does either, while the soft sign **ь** clusters at word end. Restricted distributions like these are the fingerprints of a sound system, and reading them off your own model is how you learn to see them.

WHAT YOU LEARNED

- Russian’s inventory is consonant-heavy (23 C / 10 V) because most consonants come in a plain/palatalized pair — the hard/soft contrast, written partly by the following vowel letter.
- **metrics** quantifies the system (entropy, the Zipf fit, syllable saturation, mora weight); **pairs** finds the minimal pairs that prove which sounds contrast — **мать / мять** chief among them.
- **naturalness** and functional load read an IPA-keyed feature table, so a Cyrillic **orthographic** model falls “outside the feature matrix”: for feature analysis you would build a parallel IPA model; for everything else the orthographic one serves.
- With a phonemic model you can declare **allophony** rules (**g > k / _ #**) and derive a word’s surface form with **language ipa** — Russian final devoicing turns underlying **дпут** into surface **дпук**, one rule predicting a whole class of words.
- Russian stress is **lexical** — unpredictable, mobile, contrastive (**за́мок / замо́к**) — so you record it (the lexicon **stress** field or an acute mark); **language scan** then scans a line of verse and names its metre (Pushkin’s opening is iambic tetrameter).

PART III

Words

CHAPTER 4

4

Morphology and glossing

Russian is a **fusional** language: a single ending can carry case, number and gender at once, and a noun or verb runs through a dozen forms. This is where a morphological model earns its keep — you declare the endings once, and the workbench can then both **generate** the forms and **parse** them back, and gloss whole sentences of real text.

A fragment of the noun declension

Russian nouns decline for six cases. We will not model all of it here — the point is the method, not a complete grammar — so take one masculine noun, **дом** “house”, and a couple of its endings. In the **Morphology** chapter declare the affixes and a paradigm that applies them:

```
{ morphemes: [
  { id: "gen", gloss: "GEN", form: "а", position:
"suffix" } // дом-а
  { id: "ins", gloss: "INS", form: "ом", position:
"suffix" } // дом-ом
]
paradigms: [ { name: "masc-noun", cells: [
  { features: { case: "nom" }, morphemes: [] }
  { features: { case: "gen" }, morphemes: ["gen"] }
  { features: { case: "ins" }, morphemes: ["ins"] }
] } ]
}
```

paradigm then generates the forms from the root:

```
inkhaven language paradigm Russian --root дом --template masc-
noun --gloss house
```

дом	house	(case=nom)
дома	house-GEN	(case=gen)
домом	house-INS	(case=ins)

TERM Fusional morphology

A morphology in which one affix bundles several grammatical meanings at once, and the boundaries between them are blurred — Russian **-ом** is **instrumental** and **singular** and **masculine** together, indivisibly. Contrast **agglutinative** languages like Turkish, where each meaning has its own separable suffix. Russian sits firmly on the fusional side, which is why its endings are few but each does a lot.

Reading a form backwards

The interesting direction is the reverse. Hand the parser an inflected form and it recovers the root and the ending:

```
inkhaven language parse Russian --word домом
```

```
дом 'house' + INS
```

This is analysis, not lookup: the parser strips the endings your morphology declares until what remains is a dictionary word. Point it at a page of Russian and it will tell you, form by form, what case and number each word is in — the first thing a learner (or a parser) must work out.

Interlinear glossing

The linguist's way of presenting a sentence is **interlinear glossed text**: the sentence, a gloss under each word, and a translation. `igt` builds it:

```
inkhaven language igt Russian --text "окно дома"
```

```
окно      дом-а
window    house-GEN
'the window of the house'
```

Every recognised word is glossed and, where it is inflected, segmented into its morphemes and their tags — `дома` shown as `дом-а`, `house-GEN`. The literal third line is a scaffold you replace with a real translation; words the model does not yet know pass through untouched, telling you exactly which vocabulary to add next.

TERM Interlinear glossed text (IGT)

The standard three-line format for quoting an example from any language: the morpheme-segmented sentence, a morpheme-by-morpheme gloss in standard abbreviations (GEN, INS, PL...), and a free translation. It lets a reader who does not know the language see precisely how its meaning is assembled — which is why every grammar and every paper is built from it.

You will store and reuse these glosses in Chapter 6, where a handful of them, plus a real text, become a corpus.

Exporting to standard formats

A gloss you have saved (with `igt --save`) need not stay inside Inkhaven. The `texts` command re-exports your stored interlinears for the tools linguists already use:

```
inkhaven language texts Russian --format conllu
```

`--format latex` emits a *linguex* document for a paper; `--format conllu` emits a CoNLL-U file — the interchange format of the Universal Dependencies project — with each word’s form, lemma, part of speech and grammatical features filled in from the gloss. A Russian genitive glossed `дом-а`, `house-GEN` becomes a token tagged `Case=Gen`, ready to load into the wider treebank ecosystem alongside UD’s own Russian corpora.

Agreement

Russian adjectives agree with their nouns in gender, number and case — **новый дом** “new house”, **новая книга** “new book”, **новое окно** “new window”. Once you have declared the adjective’s agreement paradigm, the Oracle can check that a form agrees:

```
inkhaven language check-agreement Russian --dependent adjective \
--form новый --root нов --head-features "gender=masc,case=nom"
```

It regenerates the ending the adjective **should** take for those features and flags the form if it differs — a plural or feminine ending on a masculine-nominative noun is caught, and the expected form named. Agreement is much of what makes Russian

sentences hang together, and the check makes the rule you declared testable against any form you write.

WHAT YOU LEARNED

- Russian is fusional — one ending carries case, number and gender together; `paradigm` generates a root's forms from the endings you declare, and `parse` reverses them, recovering root and ending from an inflected word.
- `igt` glosses a real sentence as interlinear text, segmenting each inflected word (дом-а / house-GEN) and passing unknown words through so you see what to add.
- `check-agreement` tests adjective–noun (and any head–dependent) agreement in gender, number and case against the rule you declared.

PART IV

Sentences

CHAPTER 5

5

Syntax

Sentences are built, not strung together, and syntax is the study of how. Russian is a good place to see this, because its word order is **free** in a way English's is not: the endings you met in the last chapter mark who is the subject and who the object, so the words can be rearranged for emphasis without confusion. Underneath that freedom is a stable structure, and the workbench can draw it.

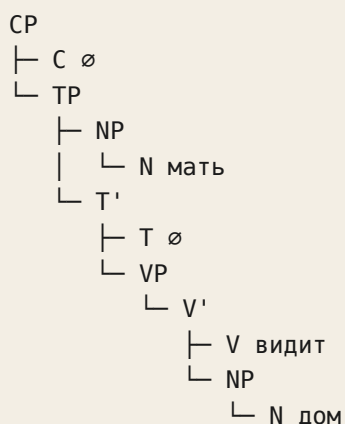
A baseline order

Russian's neutral, unmarked order is subject–verb–object, the same as English: **мать видит дом**, “mother sees house”. You declare that baseline in the grammar (`word_order: "SV0"`), and the syntax tools read it. The freedom comes from moving pieces **out** of the baseline, which we come to below.

The structure of a clause

`tree` draws the X-bar phrase-structure tree of a clause from its verb and arguments, placing heads and complements by the word order you declared:

```
inkhaven language tree Russian --verb видит --args "мать, дом"
```



The subject **мать** is the specifier of **TP**, sitting on the left; the object **дом** is the complement of the verb inside the verb phrase. This is the same skeleton every generative account of a clause assumes, drawn for a Russian sentence. Because Russian is head-initial, the verb precedes its object; a head-final language would reverse the lower branches, and asking for the tree is how you check that the order you declared builds the constituents you meant.

TERM Constituent

A group of words that behaves as a single unit — that can be moved, questioned, or replaced by one word together. The tree makes constituents visible: everything under one node is a constituent. Much of syntax is the study of which strings are constituents and which only look like they are.

Moving a piece

Russian’s word-order freedom is **movement**: a constituent lifts out of its baseline position to the front, for emphasis or to ask a question. **movement** performs that over the tree and marks the gap with a trace:

```
inkhaven language movement Russian --verb видит --args "мать,
дом" --move object
```

The object **дом** rises to the front of the clause as **NP₁**, leaving a coindexed trace **t₁** where it came from — the structure behind **дом мать видит <t>**, “the house, mother sees”. Because the case ending still marks **дом** as the object, the sentence is perfectly clear even with the words rearranged; the trace records where it is understood.

Checking a clause

The clause Oracle judges a sentence for two kinds of well-formedness at once — argument structure and agreement:

```
inkhaven language check-clause Russian --verb видит --args "мать,
дом"
```

It confirms the argument count matches the verb’s valence (a transitive verb wants two arguments, and here it has them), and — given the subject’s features and the verb’s root — that the verb agrees with its subject. A mismatch on either is reported, so the grammar you declared can be held to its own account of what a good Russian clause is.

WHAT YOU LEARNED

- Russian's neutral order is SVO; its famous freedom is **movement** out of that baseline, made possible by the case endings that mark grammatical roles.
- **t**ree draws a clause's X-bar structure (subject as specifier of TP, object as the verb's complement), placed by the declared word order; **movement** fronts a constituent and leaves a coindexed trace.
- **check-clause** judges a sentence's argument structure and subject-verb agreement together, testing the grammar against a real clause.

PART V

Usage

CHAPTER 6

6

A corpus of Russian

Everything so far has worked from a lexicon and a grammar — the language in the abstract. But a language **lived in** leaves traces: the words people actually use, in the proportions they actually use them, next to the words they actually put them near. That body of real usage is a **corpus**, and it answers questions no dictionary can. This chapter builds one from real Russian and interrogates it four ways — frequency, context, collocation, and spread.

TERM Corpus linguistics

The study of language through large bodies of real text rather than through invented examples or intuition. Its wager is that usage, counted carefully, reveals structure that introspection misses — which words are common, which go together, how meaning shifts with context. Nearly everything modern lexicography and language technology rests on was learned this way.

Assembling the corpus

A corpus in Inkhaven is drawn from two places, and you choose which with `--source:`

1. The **stored texts** — the interlinears you glossed and saved in Chapter 4 with `igt --save`. These are curated, glossed, lemma-tagged sentences.
2. The **manuscript prose** — any Russian you have written or pasted into your project. Inkhaven finds the Russian words in it the way it finds invented words in a conlanger's draft: a paragraph counts once it holds a word your lexicon knows, and within it every word that reads as Russian is gathered as a token.

`--source texts`, `--source prose`, or `--source all` (the default) select among them. For real study you want text, and Project Gutenberg holds the Russian classics in the public domain. Take the most famous opening in Russian poetry, the prologue to Pushkin's *Ruslan and Lyudmila* (1820):

У лукоморья дуб зелёный;
Златая цепь на дубе том:
И днём и ночью кот учёный
Всё ходит по цепи кругом;

Paste passages like this — several of them, from several authors — into a chapter of your manuscript, and they become one corpus. A study grows by accretion: add a Chekhov story, a chapter of Tolstoy, and every command below reads the larger whole without changing.

What is frequent?

The first question of any corpus is **how often**. **frequency** counts every word and reports the statistics real usage unlocks:

```
inkhaven language frequency Russian --source prose
```

```
texts 1 · tokens 19 · types 18 · lemmas 18 · TTR 0.95
Zipf slope -0.74 (R2 0.79)
```

```
frequency (by surface, top 20):
```

```
2  и
1  дуб
...
```

In four lines of Pushkin almost every word is unique — a **type–token ratio** near 1, the signature of dense poetry — and the little conjunction **и** “and” already leads, as function words always do. The **-- lemma** flag is where a corpus and a grammar meet: it counts a root’s inflected forms **together**, so **цень** and **цени** fall under one lemma — but only if you have declared the endings that connect them (Chapter 4). A frequency list by lemma is a frequency list of the **vocabulary**; by surface, of the **word-forms**. The gap between the two is a measure of how inflected the language is, and for Russian it is large.

TERM Zipf's law

The empirical regularity that in any large body of text a word’s frequency is roughly inversely proportional to its frequency rank — the second-commonest word occurs about half as often as the commonest, the third a third as often, and so on. Plotted on log axes it is a straight line of slope ≈ -1 . It holds across every human language and is one of the most robust quantitative facts we have; a corpus that does not show it is too small, or not language.

Point **frequency** at a whole novel and the Zipf slope tightens toward -1 and the fit **R²** toward 1. On four lines it only gestures at the curve; the law is a fact about **scale**, and it is the first thing a real corpus buys you.

A word in its contexts

Counting tells you **how much**; to see **how** a word is used, read it in context. **concordance** lines up every occurrence with its neighbours — the **KWIC** (key-word-in-context) display, the oldest tool in corpus linguistics:

```
inkhaven language concordance Russian --word цепи --window 3
```

златая цепь на дубе	[том]	и днём и
всё ходит по	[цепи]	кругом

A lexicographer reads a page of KWIC lines before writing a single definition, because a word's meaning is the sum of its contexts. Add **--lemma** and the search becomes a search for the **word**, not the **form**: a query on the lemma **цепь** gathers **цепь**, **цепи**, **цепью** and the rest into one concordance — the inflected paradigm reunited on the page. This is why the morphology of Chapter 4 matters here: without it, Russian's endings scatter each word across a dozen entries; with it, the corpus can put them back together.

The company a word keeps

Some words simply **go together** — not by grammar, but by habit. **collocations** finds them: the words that fall within a window of the target more than chance would predict, ranked by how **distinctive** the pairing is:

```
inkhaven language collocations Russian --word кот --window 4
```

The ranking is not raw co-occurrence — the commonest words co-occur with everything. It is **pointwise mutual information**, which asks how much **more** a word appears beside the target than its overall rate would predict, so a genuine partner outranks a mere frequent neighbour. Russian **кот учёный** “learned cat” is a collocation; **умный кот** “smart cat” is just two words. Collocations are much of what makes speech sound native, and a corpus is the only place to find them, because they live in usage and not in any definition.

TERM Collocation

A habitual pairing of words stronger than chance — **проливной дождь** “pouring rain”, not **сильный дождь**; **strong tea**, not **powerful tea**. Collocations are learned, not derived; they vary between languages that agree on grammar, and mastering them is much of what separates a fluent speaker from a correct one. The measure Inkhaven ranks them by, **pointwise mutual information**, is the standard association statistic.

Where a word lives in the text

A frequency count treats a text as a bag of words, but words are not spread evenly. A term that appears ten times **in one paragraph** and nowhere else is doing something different from one that appears ten times **throughout**. Reading the concordance shows this directly — the **text** column of each KWIC line tells you **which** text, and **which** paragraph, an occurrence comes from, so a word clustered in one passage stands out from one that recurs everywhere. Over a multi-text corpus this is how you separate a word that belongs to the **language** from one that belongs to a single **scene**.

FROM POEM TO LIBRARY

A four-line poem is a corpus you can read at a glance; its value here is as a demonstration. The same four questions — frequency, context, collocation, spread — scale without change to a whole Gutenberg library: drop the texts into your manuscript, and frequency, concordance and collocations read all of it. That is when the Zipf curve straightens, the collocations become real, and the statistics become something you can publish.

WHAT YOU LEARNED

- A **corpus** is real usage made countable, drawn from your stored interlinears (`--source texts`) or your manuscript prose (`--source prose`); paste public-domain Russian (Project Gutenberg) in and Inkhaven gathers its words as tokens.
- `frequency` gives word counts plus tokens, types, type–token ratio and the Zipf fit; `--lemma` counts a root’s inflected forms together (which needs the morphology of Chapter 4), turning a form-list into a vocabulary-list.
- `concordance` shows a word in every context (KWIC), `--lemma` reuniting a paradigm; `collocations` ranks a word’s habitual partners by pointwise mutual information — the idiomatic pairings a dictionary can’t give.

PART VI

Structure and History

CHAPTER 7

7

Grammar and typology

A grammar is the set of rules by which a language builds its sentences, and describing one is much of what a linguist does. Inkhaven lets you write that description down formally – the word order, how grammatical roles are marked, what each verb demands of its arguments – and then two things become possible: the analysis tools of the earlier chapters can **read** the description, and it can be **checked**, against itself and against the strong cross-linguistic generalizations of typology. This chapter is the workbench describing a grammar, with Russian as the case.

The typological profile

The broad shape of a grammar is a handful of answers, and you record them with `grammar --set:`

```
inkhaven language grammar Russian --set word_order=SV0
inkhaven language grammar Russian --set
alignment=nominative_accusative
inkhaven language grammar Russian --set adposition=preposition
inkhaven language grammar Russian --set
adjective_order=adjective_noun
inkhaven language grammar Russian --set
genitive_order=noun_genitive
```

Each answer places Russian on one axis of variation the world's languages spread across. Russian's profile is that of a typical European language — subject–verb–object, adjectives before nouns, prepositions rather than postpositions — with one system that carries most of the grammatical load: **case**.

TERM Grammatical alignment

How a language marks the core arguments of a verb — in particular, whether the single argument of an intransitive verb (**she sleeps**) is treated like the subject of a transitive verb (**she sees it**) or like its object. **Nominative–accusative** alignment, Russian's, groups the two subjects together and marks the object differently; **ergative–absolutive** alignment groups the intransitive subject with the object. It is one of the deepest divisions among the world's grammars.

In Russian the alignment is carried by the case endings of Chapter 4: the nominative marks subjects, the accusative marks direct objects, and four more cases mark everything else. A language with such rich case can afford the free word order of Chapter 5 — because the **endings** say who is subject and who is object, the **order** doesn't have to.

What each verb demands: valence

Verbs differ in how many participants they take, and the syntax tools need to know. Declare each verb's **valence** in the grammar's `verb_classes` block:

```
{ verb_classes: [
  { name: "спать", valence: "intransitive" } // sleep – one
  argument
  { name: "видеть", valence: "transitive" } // see – two
  { name: "дать", valence: "ditransitive" } // give –
  three
] }
```

Now `link`, `tree` and `check-clause` read a verb's valence rather than guessing it from the words present, so the argument-structure check of Chapter 5 can tell a genuine missing object from an intentional omission.

TERM Valence

The number of core arguments a verb requires — one for **sleep**, two for **see**, three for **give**. Valence is the backbone of a clause's structure: it determines how many noun phrases the sentence needs and what roles they play, and mismatches between a verb's valence and the arguments present are among the first things a grammar check looks for.

Russian layers **aspect** on top of valence — nearly every verb comes in an imperfective and a perfective pair (писать/написать, “to write” / “to write-to-completion”) — a grammatical category English lacks. You would record such pairs in the lexicon and note the distinction in the grammar; the point is that the description is yours to make as fine as the language demands.

Deeper parameters

Beyond the surface profile sit the abstract switches generative grammar calls **parameters**. Declare them in `ug_parameters`:

```
{ ug_parameters: {
  head_final: false    // Russian is head-initial
  pro_drop:   true     // subject pronouns may be dropped
  wh_movement: true   // question words front
} }
```

Russian earns each of these. It is **head-initial** (verbs before objects, prepositions before nouns). It is a **pro-drop** language, at least partially — **Иду домой** “(I’m) going home” needs no pronoun, because the verb ending already says who. And its question words move to the front — **Что ты видишь?** “What do you see?” — the wh-movement of Chapter 5.

TERM **Pro-drop**

The property of a language that lets it omit a subject pronoun when the verb’s agreement already identifies the subject — Italian **parlo** “(I) speak”, Russian **иду** “(I) go”. Pro-drop correlates with rich verb agreement: a language whose verb endings spell out the subject can afford to leave the pronoun unsaid. English, with almost no agreement, cannot.

Checking the description

A formal description can be **inconsistent** — you might declare a head-final parameter while setting an SVO (head-initial) word order. **grammar-check** catches such conflicts:

```
inkhaven language grammar-check Russian
```

It validates the typed blocks and cross-checks them against each other and against the typological answers — flagging, for instance, a **head_final** parameter that contradicts your declared word order. Run against a language you know, it is a proof-reader for your model; the errors it finds are errors in the **description**, caught before they mislead you.

Testing against the universals

Typology’s central discovery is that grammatical features do not vary independently. They cluster, and the clusters are stated as **implicational universals**: “if a language has X, it (tends to) have Y”. `universals` checks a profile against the classic ones:

```
inkhaven language universals Russian
```

It reports the language’s **head-directionality harmony** — whether it consistently puts heads before their complements — and judges the Greenberg and Dryer implicational universals satisfied, violated, or not applicable. Russian’s profile lines up: a verb–object language **tends** to be prepositional (Russian is), to put genitives after the noun (**крыша дома**, “roof of-the-house”), and to place relative clauses after their head — and Russian obliges on each. That harmony is not a coincidence; it is the pressure typology measures.

TERM Implicational universal

A conditional generalization over the world’s languages: **if** a language has one feature, **then** it almost always has another — if the object follows the verb, the language tends to use prepositions rather than postpositions. Such universals are the strongest empirical results typology has produced, and checking a language against them tests whether your description of it is internally coherent.

A VIOLATION IS A BUG IN THE MODEL

If `universals` flagged a violation for Russian, that would not be a discovery about Russian — it would be an answer set wrong in your model. That is exactly the check’s value: run against a language you know conforms, it tells you whether your **formal description** is consistent before you trust it on a language you don’t. A model that breaks the universals a real language obeys is a model to fix.

WHAT YOU LEARNED

- `grammar --set` records a language's typological profile (word order, alignment, adpositions, adjective and genitive order); Russian is a nominative-accusative, head-initial, prepositional language whose case system carries the grammatical load.
- `verb_classes` declares each verb's **valence** (intransitive / transitive / ditransitive) and `ug_parameters` its deeper switches (head-final, pro-drop, wh-movement) — both read by the syntax tools of Chapter 5; `grammar-check` validates the description for internal consistency.
- `universals` tests the profile against head-directionality harmony and the classic implicational universals; run against a language you know, it validates the model — a violation where the real language conforms means the description is wrong.

CHAPTER 8

8

Historical linguistics

Languages are the visible present of a long past. Russian did not always sound as it does now; the modern forms are the outcome of centuries of regular sound change, and the comparative method — reasoning backward from the daughters to their common ancestor — is one of the great achievements of the human sciences. This final chapter turns the workbench on the past, and treats the study of it as what it is: a science of **hypotheses**.

A famous change

Old East Slavic had two short vowels, written ъ and ѣ and called the **yers**. Around the twelfth century the weak yers were lost, and the strong ones became full vowels — the single most consequential sound change in Russian’s history. Old **сѣнь** became modern **сон** “dream”; **дѣнь** became **день** “day”. Whole classes of words were reshaped by one regular rule.

That rule is a **claim** about Russian’s history, and a claim is something to record and test, not merely assert. Register it:

```
inkhaven language hypothesize Russian --kind sound-change \
  --claim "ѣ > ъ / _ #" --note "loss of the weak final yer" --id
  yer-loss
```

It enters the language’s register of hypotheses, marked **proposed**.

TERM Sound change

A regular alteration in pronunciation that spreads through a language’s whole vocabulary — not a one-off, but a rule: **every** word in the right environment changes. The regularity is what makes historical linguistics possible; because sound change is systematic, its effects can be predicted, reversed, and used to prove that two languages descend from one.

Testing the consequences

A hypothesis earns its keep by making predictions. **hypothesis-check** runs your sound-change claim through the Consequence Tracer — applying it across the lexicon and reporting exactly which words it changes and which distinct words it **merges** into homophones:

```
inkhaven language hypothesis-check Russian --id yer-loss
```

Now the claim is falsifiable. If the words it predicts to change are the ones that did, and the mergers it predicts are ones Russian actually shows, the evidence supports

it; if it collapses distinctions the language in fact keeps, that is evidence against. Record the verdict:

```
inkhaven language hypothesis-status Russian --id yer-loss --
status supported
```

inkhaven language hypotheses Russian then shows the register at a glance, each claim marked with its standing — proposed, supported, refuted, retired. A refuted hypothesis kept on the books is as valuable as a supported one: it stops you proposing it again.

TERM The comparative method

The technique of reconstructing an unattested ancestor by systematically comparing its descendants: line up **cognates** — words descended from one original — find the regular sound correspondences between them, and reason back to the form that would yield them all. It is how Proto-Indo-European was reconstructed from Sanskrit, Greek, Latin and the rest, without a single written record of it surviving.

Russian among the Slavic languages

Russian is one daughter of several. Its sisters — Ukrainian, Belarusian, Polish, Czech, the South Slavic languages — descend with it from Proto-Slavic, and the regular differences between them are the raw material of reconstruction. Where Russian has **город** “city”, Polish has **gród** and Old Church Slavonic **градъ**: one cognate set, three reflexes of a single ancestral form, and the **pleophony** (the Russian **-opo-** against South Slavic **-pa-**) is itself a regular, datable change.

Model the sisters as a language family and Inkhaven’s diachronic tools apply directly: **cognates** traces a proto-form’s reflex in each daughter, **reconstruct** proposes an ancestor from a cognate set, and each proposal is another hypothesis for the register. The comparative method, systematized: propose a change or a cognacy, trace its consequences, check it against the forms, and write down whether it held.

WHAT YOU LEARNED

- Russian is the outcome of regular sound change — the fall of the yers turned **сънь** into **сон**; such changes are rules over the whole vocabulary, which is what makes them recoverable.
- **hypothesize** records a sound-change (or cognacy, or borrowing) claim; **hypothesis-check** runs it through the Consequence Tracer to make it falsifiable; **hypothesis-status** records the verdict as the evidence comes in.
- Russian's Slavic sisters supply the cognate sets of the comparative method; **cognates** and **reconstruct** reason across a modelled family, each result another hypothesis to test — historical linguistics as an explicit, testable loop.

APPENDIX A



The research workflow

Every command used in this book, grouped by the stage of study it belongs to. Most follow the shape `inkhaven language <action> <language> [options];add --help` to any command for its full signature. Throughout, `<lang>` is the language you are studying — `Russian`, in this book.

Modelling the language

`init <name>` Create the language and its (empty) chapters.

stats <lang> [--json] A descriptive profile — inventory size, average word shape.

add-word <lang> <word> --type POS --translation M Add a lexicon entry (or `--import file.csv`).

query <lang> [--pos] [--text] [--json] Search the lexicon.

The phoneme inventory and morphology are declared as HJSON blocks in the language's Phonology and Morphology chapters (Chapters 2 and 4); see the companion book's HJSON reference for the full schema.

The sound system

metrics <lang> [--json] Quantitative sound-system metrics — entropy, the Zipf fit, phonotactic saturation, mora weight.

pairs <lang> [--limit N] [--json] Minimal pairs and the feature each turns on (feature naming needs an IPA-based model — Chapter 3).

naturalness <lang> [--json] Score the inventory against cross-linguistic tendencies (IPA-based; see Chapter 3).

distribution <lang> [--json] Where each segment appears (onset / nucleus / coda, word edges) and any restricted distributions.

Words and sentences

paradigm <lang> --root R --template T --gloss G Generate a word's inflected forms from the declared endings.

parse <lang> --word W [--json] Analyse a surface form into root + affixes (the inverse of `paradigm`).

igt <lang> --text "..." [--save --name N] [--json] Interlinear glossed text — the segmented sentence, its gloss, a literal translation; `--save` keeps it in the `Texts` chapter.

check-agreement <lang> --dependent D --form W --root R --head-features "..." Does a dependent (adjective, ...) agree with its head's features?

tree <lang> --verb V --args "subj,obj" [--word-order 0] [--json] The X-bar phrase-structure tree of a clause.

movement <lang> --verb V --args "..." --move ROLE Front a constituent, leaving a coindexed trace.

check-clause <lang> --verb V --args "..." [--subject-features "..."] The clause Oracle — argument structure and subject–verb agreement.

The corpus

frequency <lang> [--source texts|prose|all] [--lemma] [--top N] [--json]

Word-frequency list and statistics (tokens, types, TTR, Zipf) over the corpus.

concordance <lang> --word W [--source ...] [--lemma] [--window N] [--json] A

KWIC concordance — every occurrence of a word with its context.

collocations <lang> --word W [--source ...] [--lemma] [--window N] [--json] A

word's collocates, ranked by co-occurrence and PMI.

texts <lang> [--name N] [--set-translation "..."] [--format text|latex] List, print, curate or export the stored interlinear texts.

Typology and history

grammar <lang> [--set feature=value] View or set the typological profile.

universals <lang> [--json] Check the profile against head-directionality harmony and the Greenberg/Dryer implicational universals.

hypothesize <lang> --kind sound-change|cognacy|borrowing --claim "..." [--evidence "..."] [--id N]

Record a diachronic/comparative hypothesis.

hypotheses <lang> [--status S] [--json] List the hypothesis register.

hypothesis-check <lang> --id N Run the Consequence Tracer over a sound-change hypothesis's claim.

hypothesis-status <lang> --id N --status S Move a hypothesis along as the evidence comes in.

cognates <proto> --form W · **reconstruct** --forms "..." Trace a proto-form's reflexes, or reconstruct an ancestor from a cognate set.

The companion

inkhaven linguistic [--language L] Open the full-screen Linguistic companion over your languages — a grounded chat plus the slash-commands `/parse`, `/tree`, `/clause`, `/igt`, `/frequency`, `/kwic`, `/coll`, `/hypotheses`, `/hcheck`, each running the tool above inline.

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. It is not an accident that a tool he built for writers carries the same instinct into every kind of book it helps make, and into the linguistic analysis this one describes:

never a result the model cannot account for.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not small utilities, but programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistackvm`, `bundcore` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work. Inkhaven's linguistics grew the same way: an engine for inventing languages that turned out, with no extra machinery, to be an engine for studying them.

A work of love

Inkhaven is open source, under a permissive licence — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking the licence also lets you sell it; the author would, gently but firmly, disagree with your doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the people who can least afford to pay for software — the novelist on a battered laptop, the graduate student writing a thesis, the field linguist documenting a language with no budget at all — and turning it into a commercial product would betray the reason it exists. It carries no analytics, no telemetry, no upsell; the binary will never phone home.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet. If this book helps you understand a language a little better — your own, or one you are documenting for the first time — that is enough.

Where to find more

GitHub `@vulogov` — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and pull requests welcome.

LinkedIn `/in/vladimirulogov` — posts on observability, the occasional long-form essay.

YouTube `@vulogov` — talks and walkthroughs from the conference trail.

If the tool ever gets in the way of the language instead of out of it, open an issue on GitHub. The author reads them.

E N D O F T H E B O O K